02

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

공연 예매 플랫폼

제1부 기반: 백엔드와 Node.js 런타임 · 1장 오리엔테이션

오픈 1초에 평소의 수백 배, 서버가 먼저 답해야 할 질문



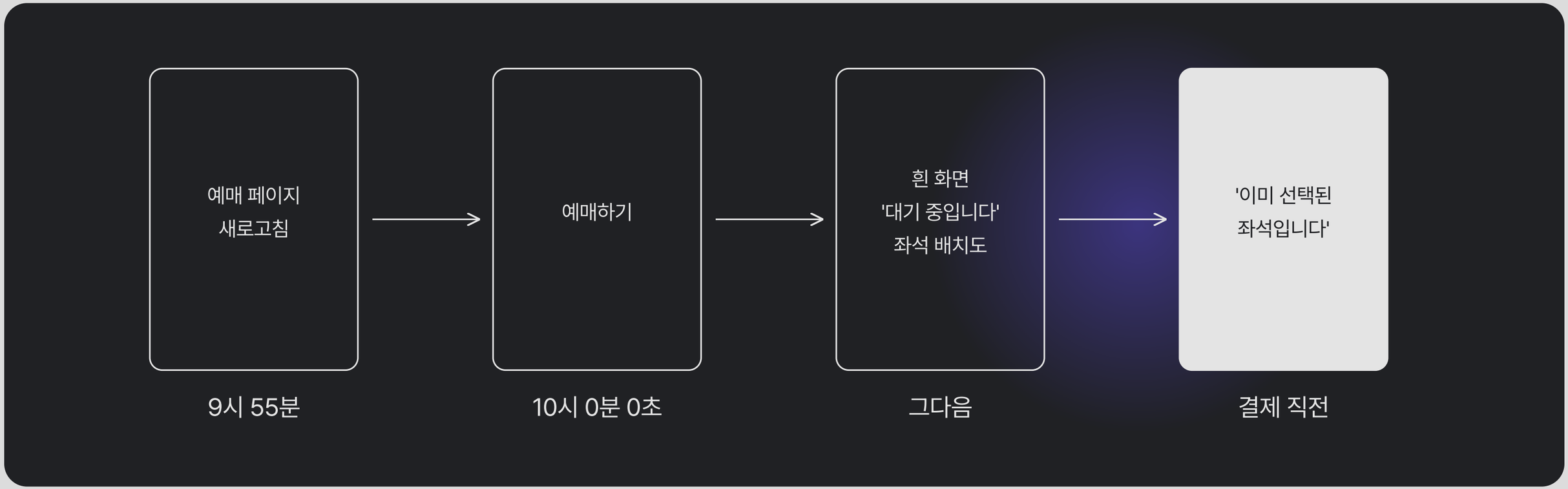
© 한상훈 NestJS 백엔드 마스터

오전 10시의 티켓팅 화면

→ 9시 55분: 예매 페이지 열고 새로고침 / 10시 0분 0초: 예매하기

→ 흰 화면 / '접속자가 많아 대기 중입니다' / 좌석 배치도

→ 결제 직전 '이미 선택된 좌석입니다'



© 한상훈 NestJS 백엔드 마스터

좌석 5만 석, 접속자 50만 명

9시 59분까지

공연 정보 조회 몇 건

10시 0분 0초

1초 안에 평소의 수백 배

50만 명

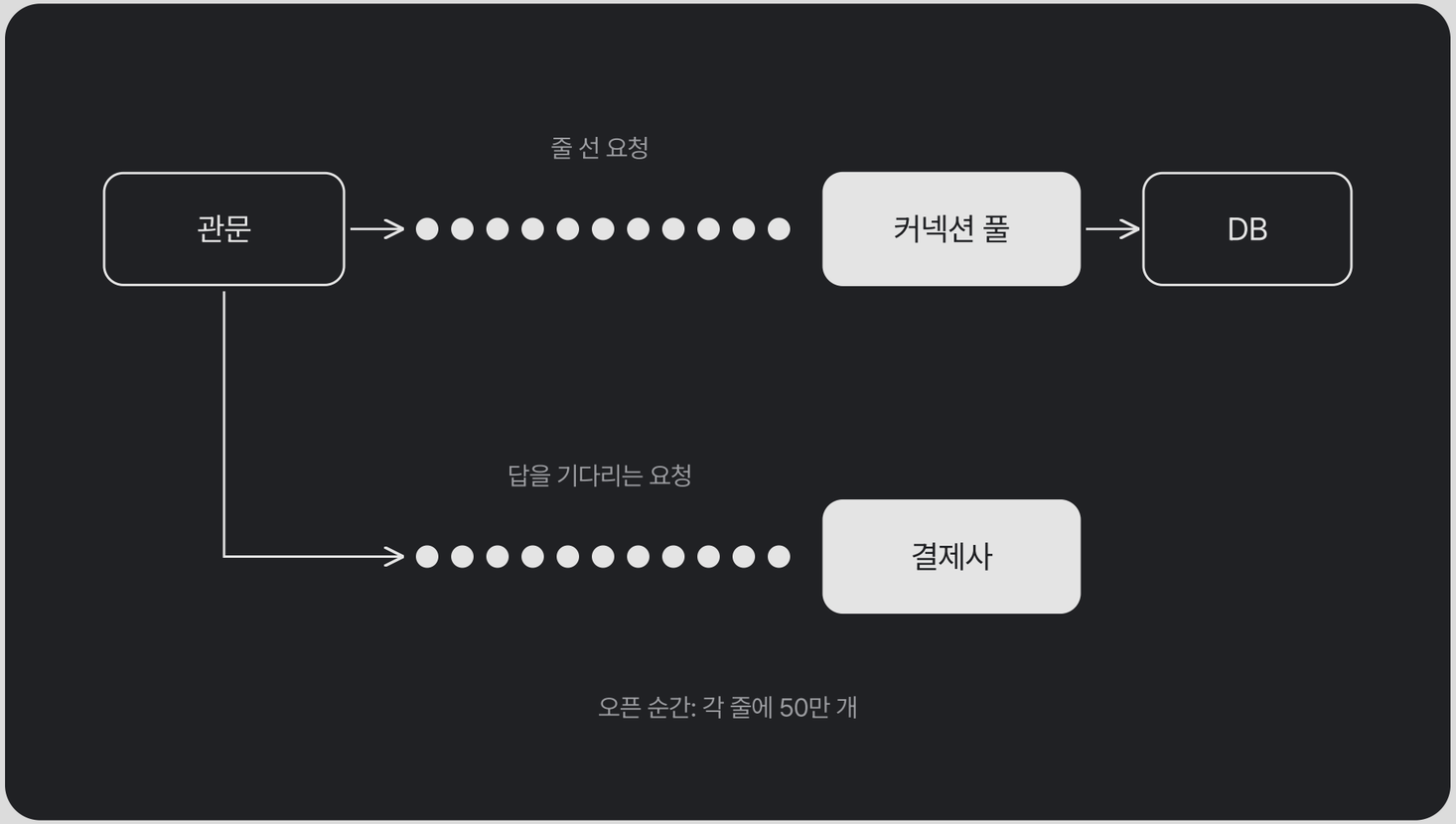
시계를 보다가 같은
1초에 도착

데이터 블록: 좌석 수와 접속자 수는 강의용 가정값

© 한상훈 NestJS 백엔드 마스터

가장 느린 곳 앞의 50만 개

"요청 하나는 여러 곳을 거치고, 그중 가장 느린 곳이 응답 시간을 정합니다."



→ 평소: 가장 느린 곳의 줄도 금방 해소

→ 오픈 순간: 커넥션 풀 앞, 결제사 앞에 50만 개

© 한상훈 NestJS 백엔드 마스터

이 플랫폼을 고른 이유

구분	회원 가입·게시판	공연 예매
요청 분포	하루 종일 고르게	오픈 1초에 집중
로컬 감각	그럭저럭 맞음	세 개가 같은 1초에 깨짐
버그	하루 한두 번	오픈 첫날 전부

© 한상훈 NestJS 백엔드 마스터

평소와 오픈 순간

구분	초당 요청
평소	1,000개
오픈, 한 번씩 클릭	50만 개 (500배)
오픈, 페이지당 API 4개	200만 개

데이터 블록: 전부 계산용 가정값 (실제 예매처 수치 아님)

© 한상훈 NestJS 백엔드 마스터

예매 페이지 하나가 부르는 API

API 1

로그인 확인

API 2

회차 정보

API 3

좌석 현황

API 4

공지 사항

→ 버튼 한 번 = 요청 여러 개

© 한상훈 NestJS 백엔드 마스터

하루 평균의 착시

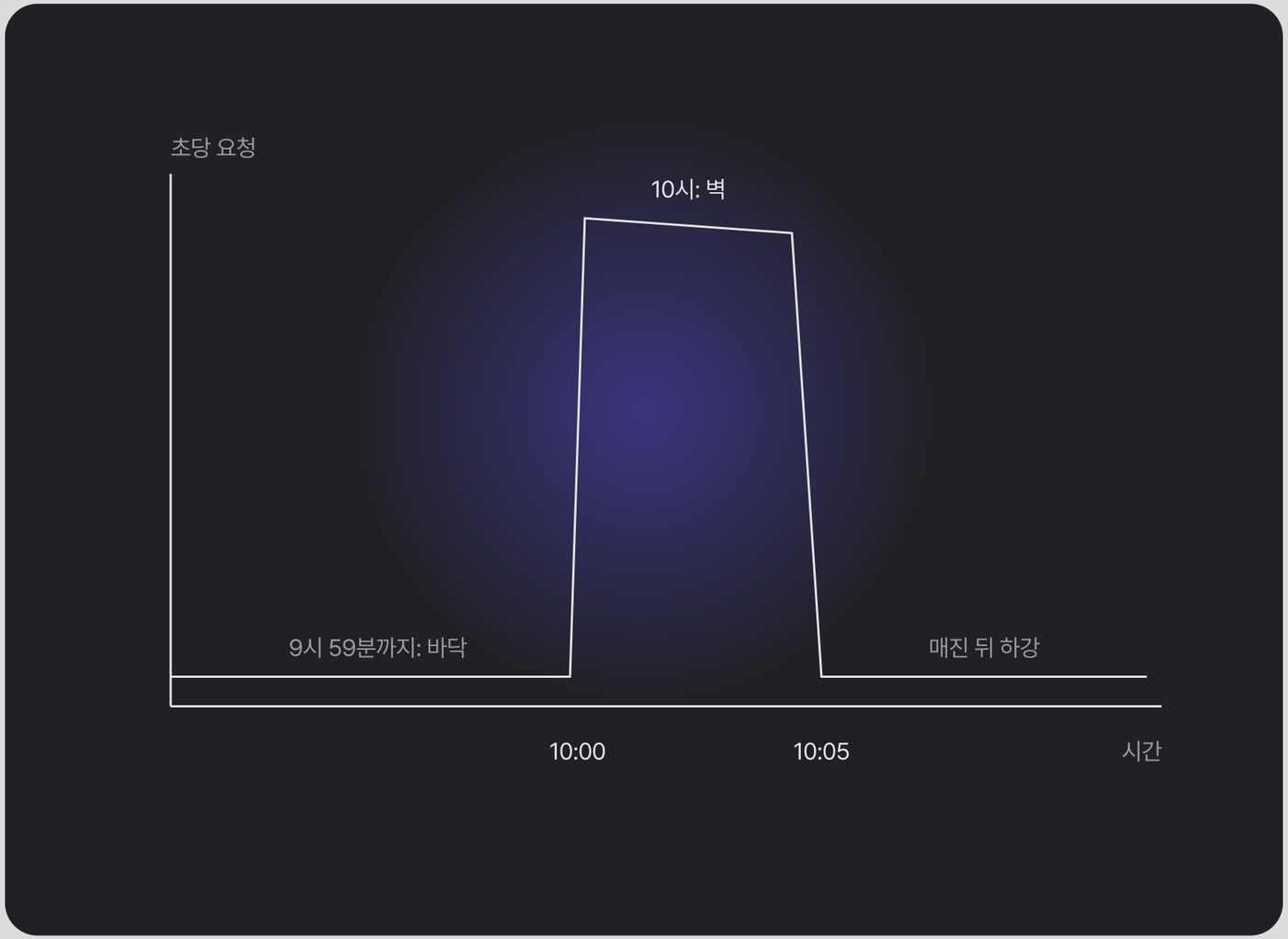
항목	값
하루	86,400초
평소 하루 요청	8,640만 개
오픈 1초 추가 뒤 하루 평균	초당 약 1,006개

→ 평균으로 보면 한가한 서비스

→ 평균 기준 서버: 그 1초에 붕괴

© 한상훈 NestJS 백엔드 마스터

9시 59분과 10시의 트래픽



→ 9시 59분까지: 바닥

→ 10시: 벽처럼 상승

→ 10시 5분쯤: 매진 뒤 하강

© 한상훈 NestJS 백엔드 마스터

가장 바쁜 1초 기준 설계

숫자

서버 대수, 커넥션 풀 크기, 결제사 초당 건수

정하는 법

전부 가장 바쁜 1초에서 역산

평소

대부분 유휴

→ 식당: 평일 오후 3시 말고 토요일 저녁 7시 손님 수로 테이블과 직원 수 결정

© 한상훈 NestJS 백엔드 마스터

미리 알려진 1초

1

오픈 시각: 몇 주 전 공지

2

미리 서버 증설 가능

3

몰리는 요청을 줄 세울
준비 가능

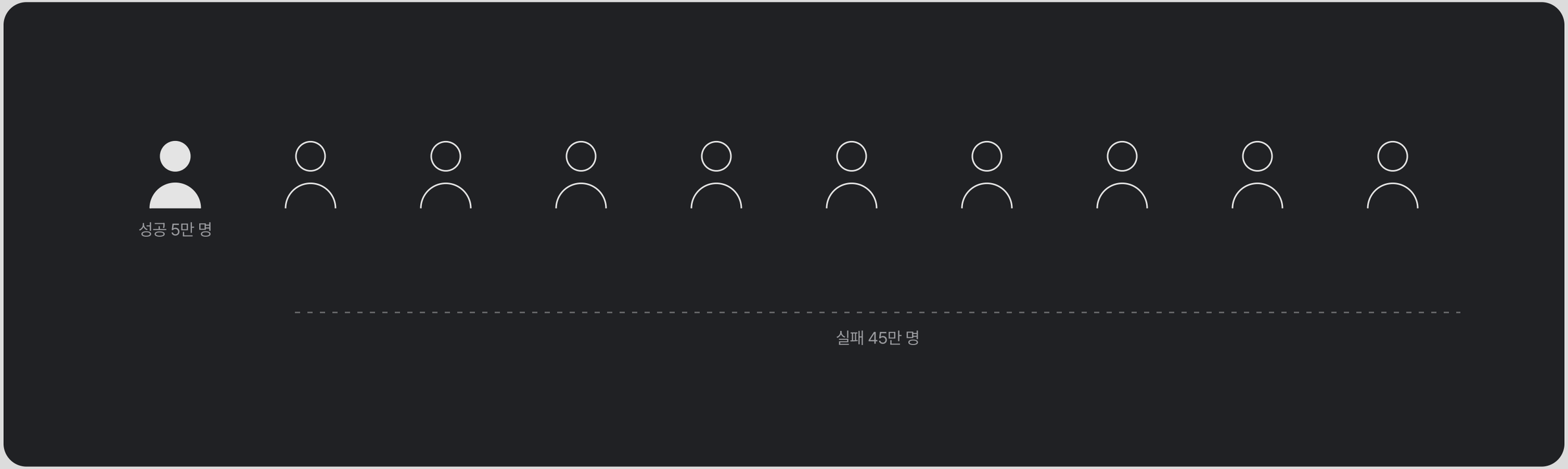
© 한상훈 NestJS 백엔드 마스터

5만 명과 45만 명

→ 좌석 5만 석, 1인 1매 → 성공 5만 명

→ 나머지 45만 명: 반드시 실패

→ 오픈 순간 응답의 90%: 실패 응답



© 한상훈 NestJS 백엔드 마스터

실패 응답도 기능

가장 많이 나가는 응답

실패

잘 만든 실패

빨리, 정확하게

© 한상훈 NestJS 백엔드 마스터

30초 뒤의 매진, 1초 뒤의 매진

매진 응답 시각	서버 안에 남는 것
30초 뒤	45만 개 요청의 연결과 메모리, 30초 동안
1초 뒤	1초 만에 해제

→ 늦은 실패: 성공할 5만 명의 요청까지 지연

© 한상훈 NestJS 백엔드 마스터

매진을 알기까지

빠른 매진 응답의 조건

서버가 먼저 빨리 매진을 앎

확인마다 DB 왕복

45만 명의 '매진 맞나요'가
전부 DB로

비용

실패 응답 하나에도 비용

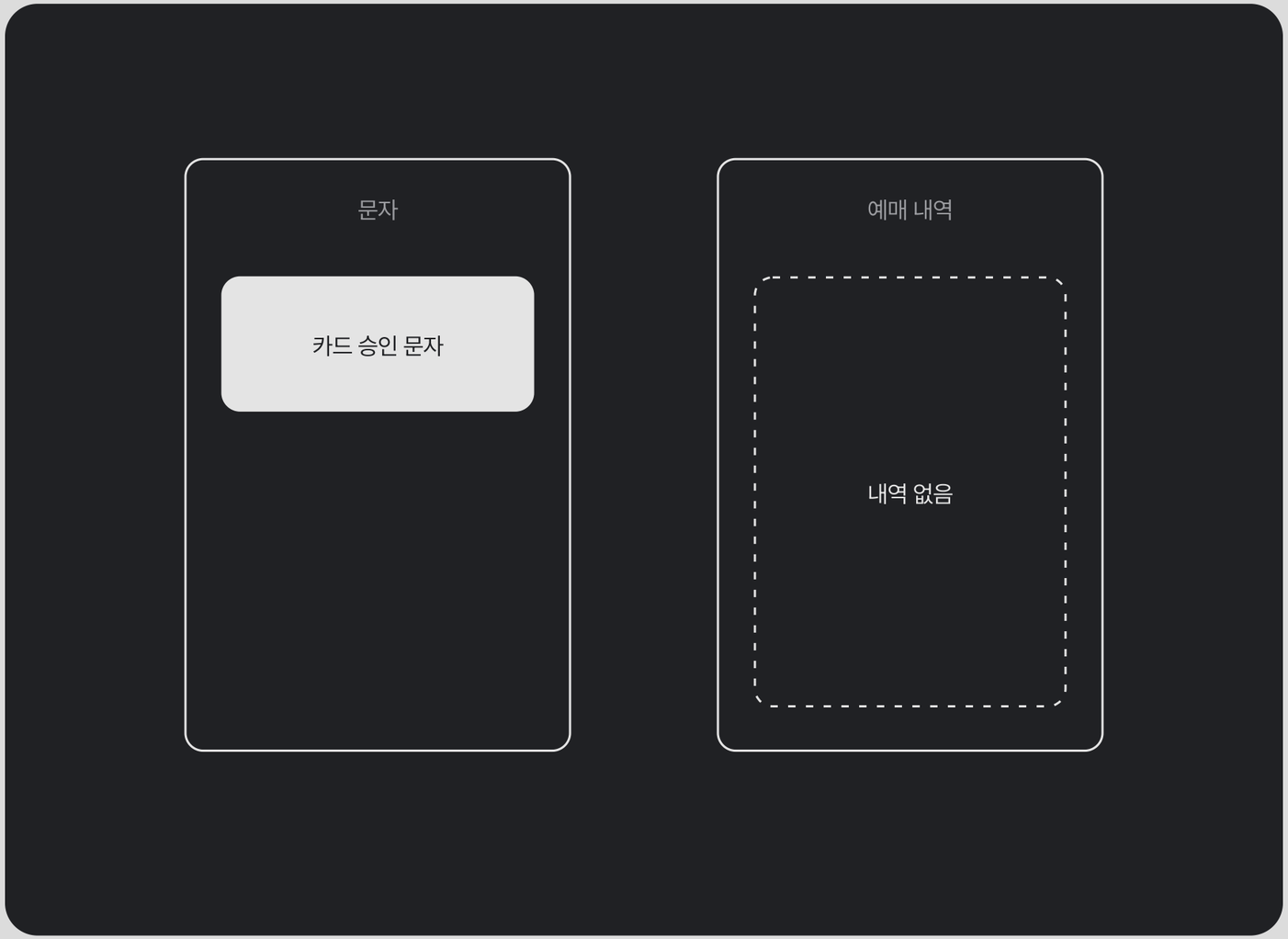
© 한상훈 NestJS 백엔드 마스터

실패 문구와 다음 1초

화면 문구	손님 행동	다음 1초
'잠시 후 다시 시도해 주세요'	1초마다 새로고침	45만 개
'전석 매진'	대부분 창 닫기	급감
'오류가 발생했습니다'	페이지 전체 새로고침	요청 여러 개
'선택하신 좌석이 방금 판매되었습니다, 다른 좌석을 골라 주세요'	다른 좌석만 다시 선택	요청 하나

© 한상훈 NestJS 백엔드 마스터

돈은 빠졌는데 좌석이 없는 손님



→ 결제 승인 → 그사이 좌석이 다른 사람에게

→ 카드 문자는 왔는데 예매 내역 없음

→ 고객센터 전화 → 환불 처리 → 다음 공연 이탈

→ 느린 실패보다 비싼 실패

© 한상훈 NestJS 백엔드 마스터

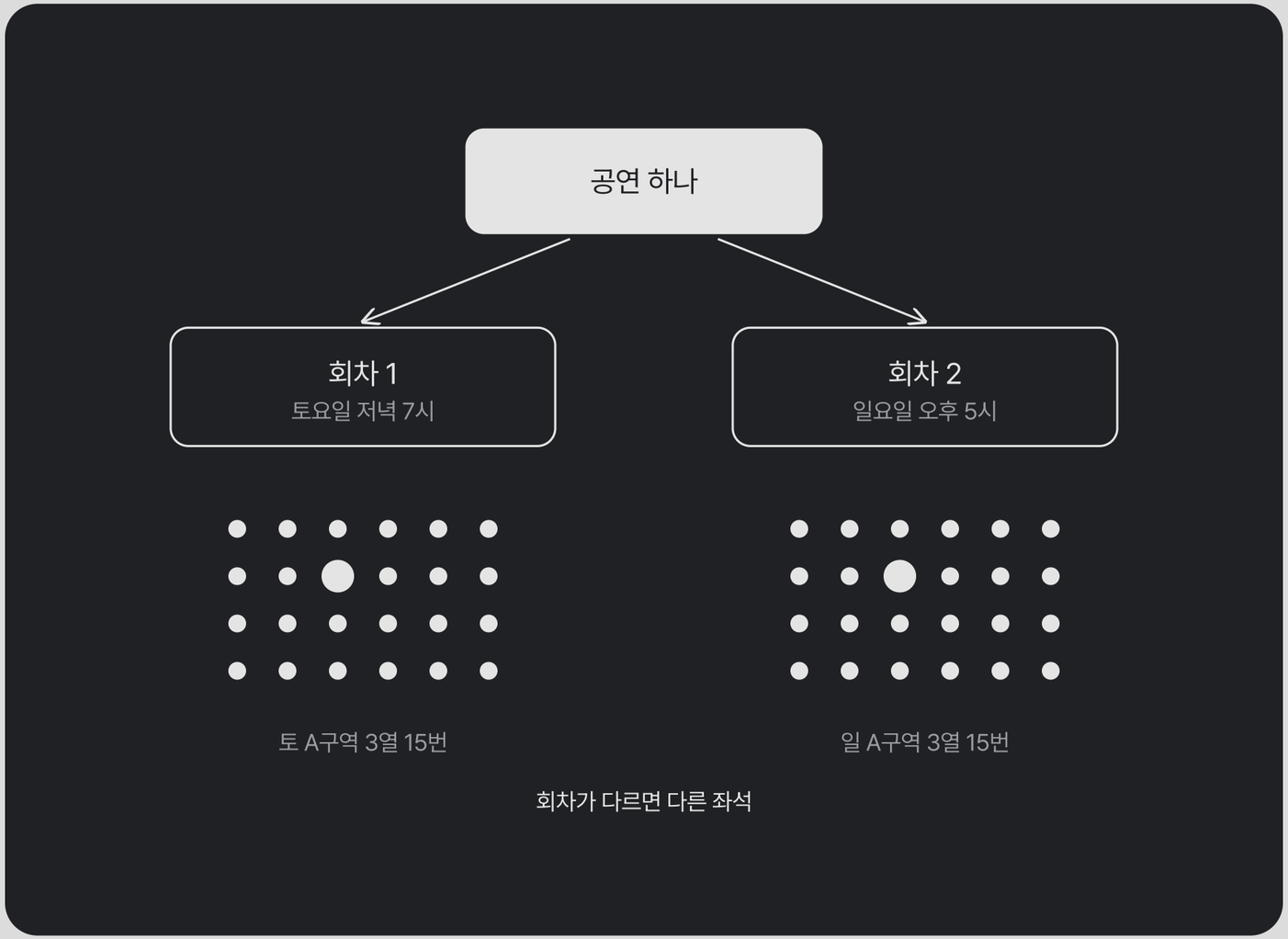
공연, 회차, 좌석

→ 공연 하나, 회차 둘 (토요일 저녁 7시, 일요일 오후 5시)

→ 좌석: 회차마다 따로

→ 토요일 A구역 3열 15번 ≠ 일요일 A구역 3열 15번

→ 좌석 하나 = 회차 + 구역 + 열 + 번호



© 한상훈 NestJS 백엔드 마스터

기본 단어 여섯 개

1

공연

2

회차

3

좌석

4

예매

누가, 어느 회차의, 어느 좌석을
샀는지의 기록

5

결제

6

알림

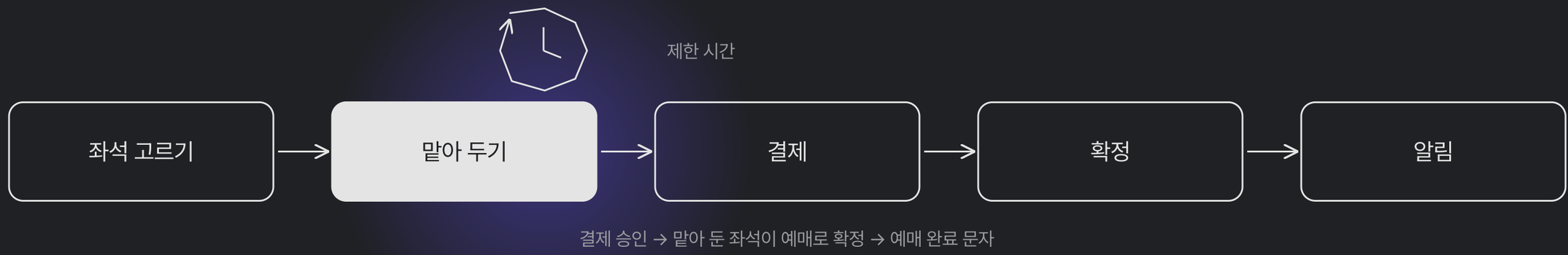
→ 코드에서도 그대로 모듈 이름

© 한상훈 NestJS 백엔드 마스터

예매 흐름 다섯 단계

→ 좌석 고르기 → 맡아 두기(제한 시간) → 결제 → 확정 → 알림

→ 맡아 두기: 고른 순간부터 결제까지 다른 사람의 선택 차단



결제 승인 → 맡아 둔 좌석이 예매로 확정 → 예매 완료 문자

© 한상훈 NestJS 백엔드 마스터

단계마다 다른 실패

단계	실패 모양
좌석 고르기	남이 먼저 선택
맡아 두기	제한 시간 초과
결제	결제사 지연, 무응답
확정	결제는 됐는데 좌석이 이미 풀림
알림	문자 발송 지연

© 한상훈 NestJS 백엔드 마스터

테이블보다 목록 먼저

테이블부터 그리면

충돌과 실패를 보기 전에
구조 고정

테이블이 생긴 뒤의 생각

테이블에 맞춰짐

먼저 만들 것

목록 세 개

© 한상훈 NestJS 백엔드 마스터

목록 하나: 동시 요청이 부딪히는 곳

같은 데이터를 두 요청이 같은 순간에 고치려는 곳

© 한상훈 NestJS 백엔드 마스터

1밀리초 차이의 같은 좌석

→ 10:00:00.120 나, A구역 3열 15번

→ 10:00:00.121 다른 손님, 같은 좌석

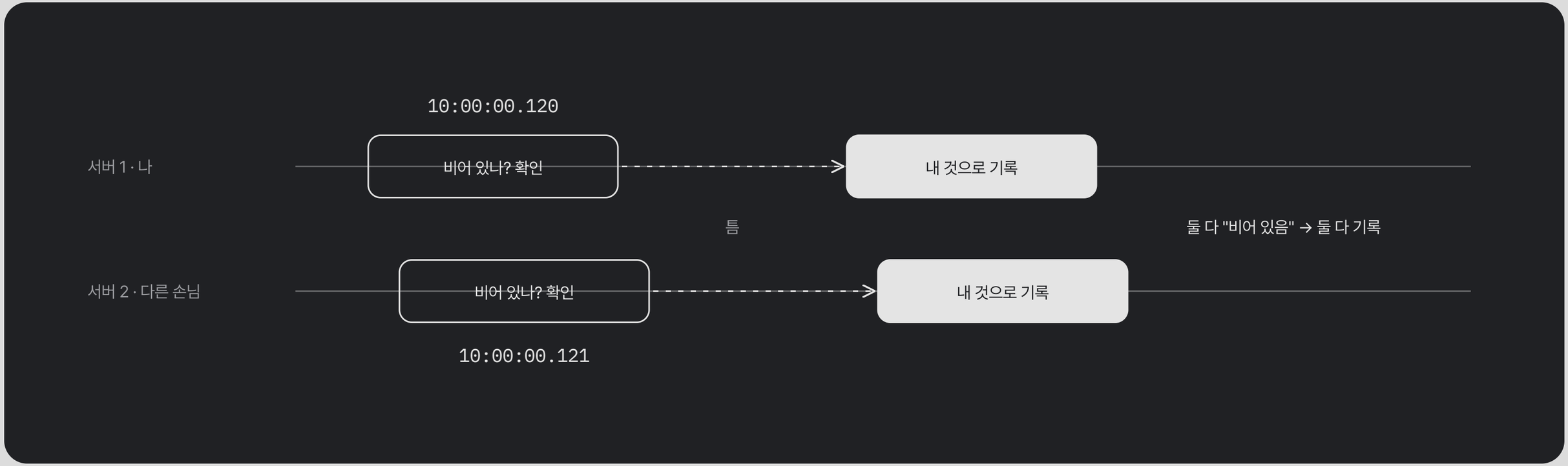
→ 로드 밸런서 → 서로 다른 서버

→ 서버 코드: "비어 있나?" 확인 → "내 것으로 기록"

→ 둘 다 "비어 있음" 확인 → 둘 다 기록

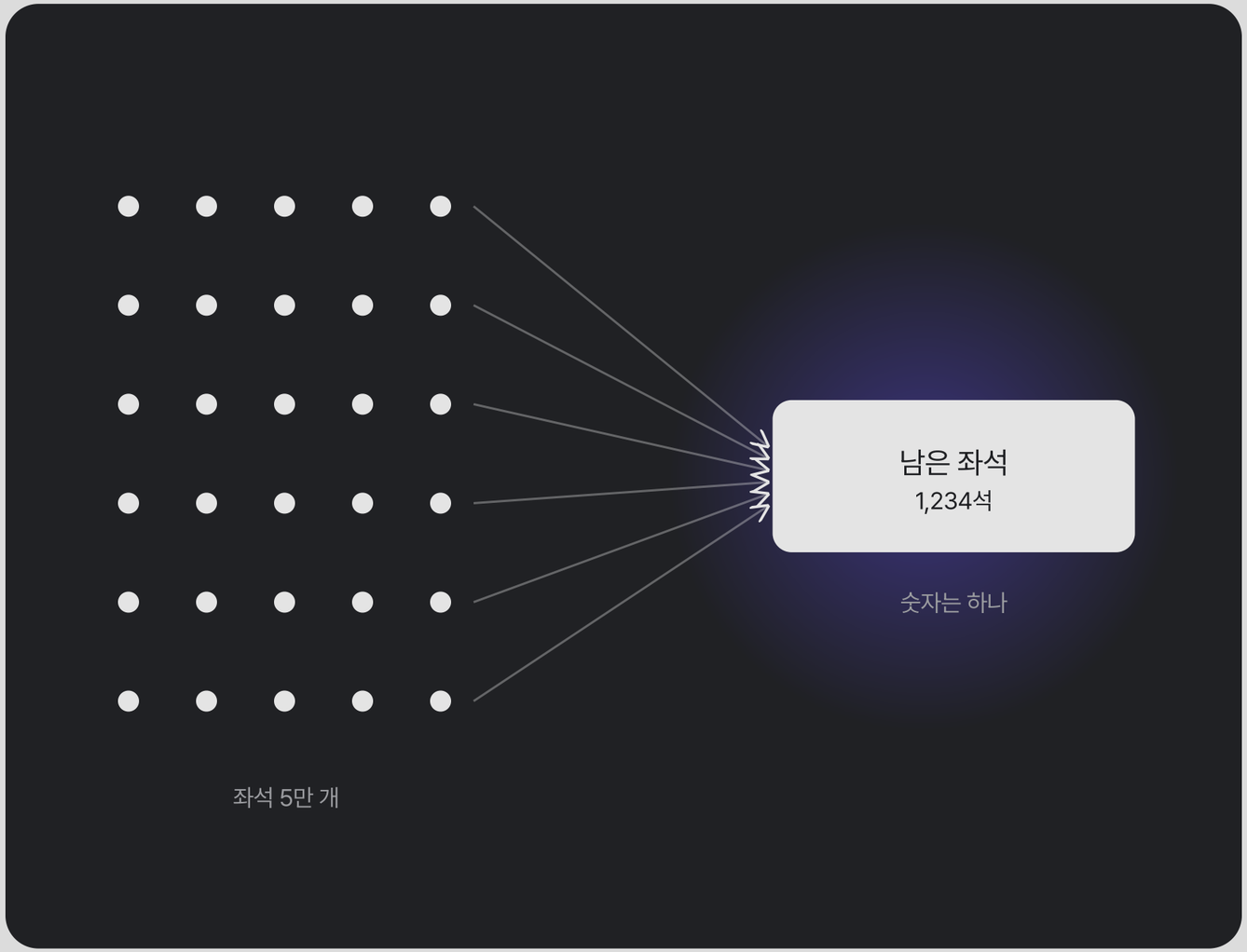
→ 확인과 기록 사이의 틈, 50만 명이면 누군가는 그 틈으로

→ 로컬: 한 번 찍고 결과 보고 다음 선택 → 테스트 통과



© 한상훈 NestJS 백엔드 마스터

남은 좌석 1,234석



→ 예매 화면 위 '남은 좌석 1,234석'

→ DB에 숫자 하나, 예매마다 1씩 차감

→ 좌석은 5만 개로 분산, 이 숫자는 하나 → 한 줄로 집중

→ 둘 다 1,234 읽기 → 둘 다 1,233 기록 → 두 장 팔리고 숫자는 하나만 감소

© 한상훈 NestJS 백엔드 마스터

같은 손님의 결제 두 번

1

결제 버튼 → 화면 정지 → 한 번 더

2

같은 손님, 같은 좌석, 결제 요청 둘

3

서로 다른 서버에서 각자 결제사 호출 → 결제 두 번 가능

© 한상훈 NestJS 백엔드 마스터

목록 하나의 첫 버전

부딪히는 곳	부딪히는 이유
같은 좌석	확인과 기록 사이의 틈
남은 좌석 수	숫자 하나를 수만 요청이 동시에 수정
같은 손님의 결제	다시 누른 버튼, 다른 서버의 같은 결제

→ 막는 방법은 방법마다 비용이 달라 따로 검토

→ 적지 않은 곳: 오픈 날 손님 항의로 처음 발견

© 한상훈 NestJS 백엔드 마스터

목록 둘: 외부 호출과 실패 시 동작

우리 서버가 남의 서버를 부르는 곳 전부
실패하면 우리가 할 일

© 한상훈 NestJS 백엔드 마스터

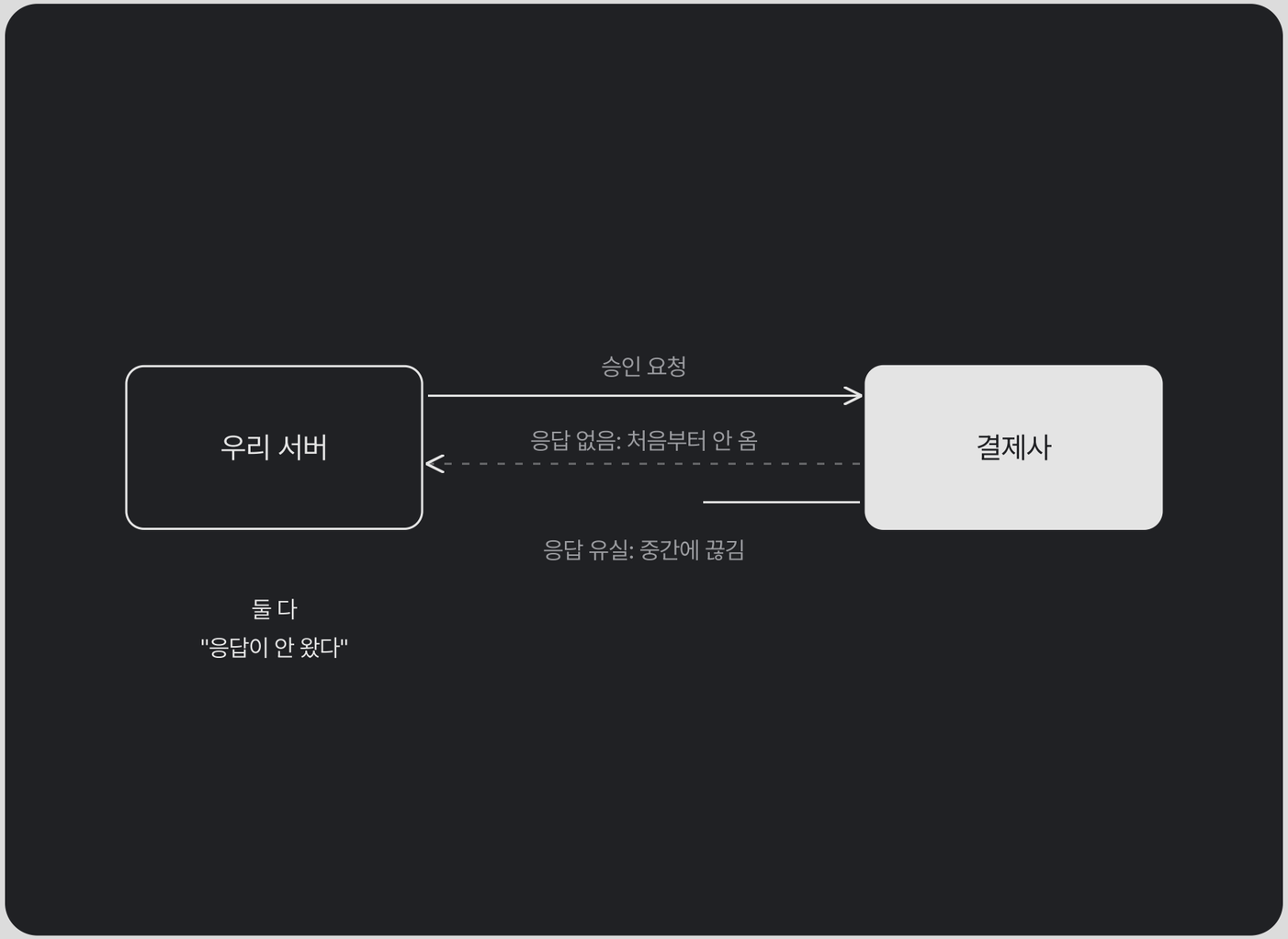
결제사가 실패하는 세 모양

모양	우리 서버가 보는 것	실제 결제
답이 없음	응답 안 옴	안 됨
느림 (0.5초 → 5초, 10초)	늦은 응답, 쌓이는 대기	됨 또는 안 됨
성공 뒤 응답 유실	응답 안 옴	됨

→ 느림도 실패: 쌓이는 요청, 다시 누르는 결제 버튼

© 한상훈 NestJS 백엔드 마스터

응답 없음과 응답 유실



→ 우리 서버에서는 둘 다 "응답이 안 왔다"

→ 좌석을 풀면: 돈은 빠졌는데 좌석이 없는 손님 가능

→ 좌석을 잡고 있으면: 결제 안 된 좌석이 계속 묶임

© 한상훈 NestJS 백엔드 마스터

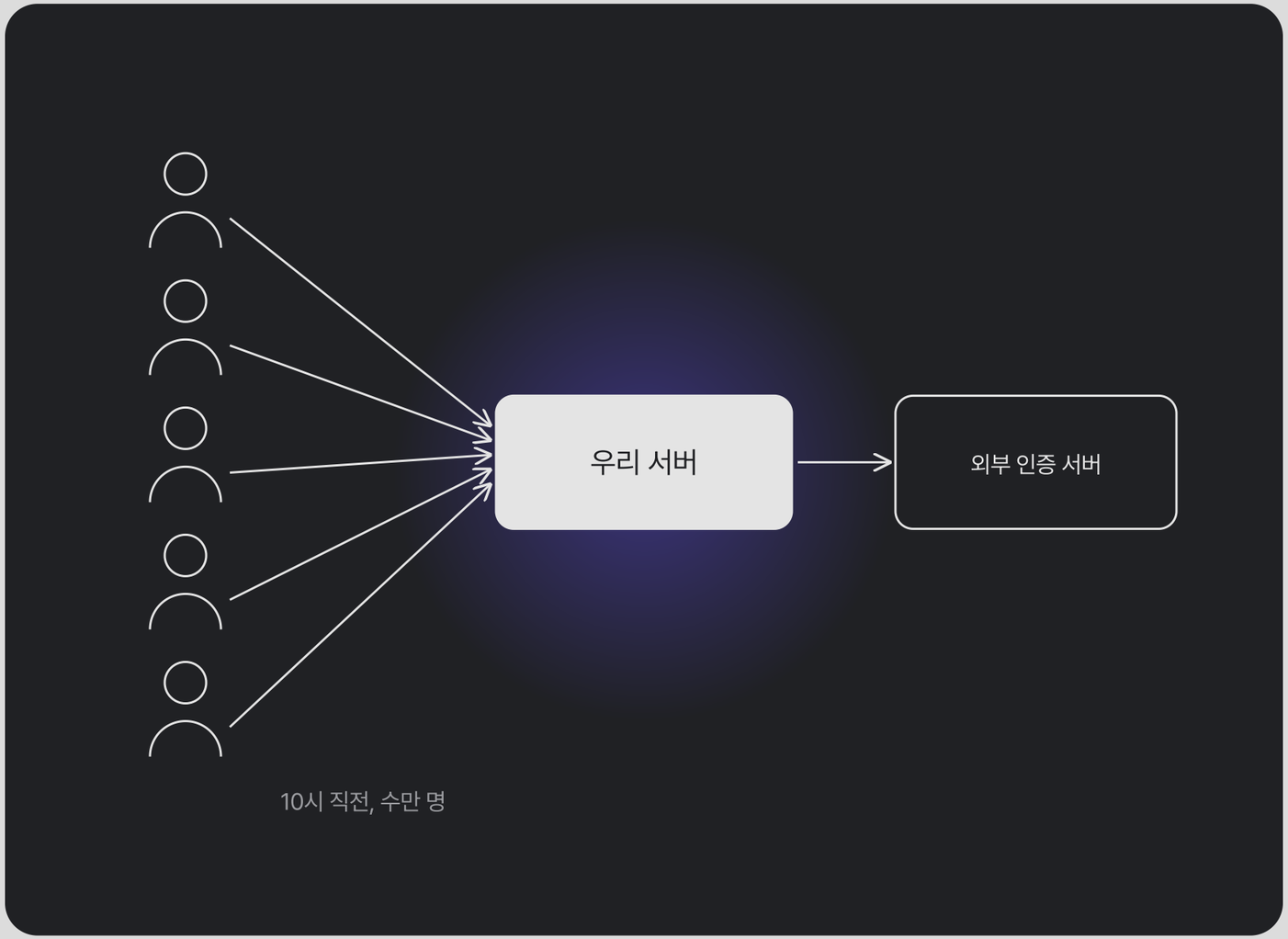
소셜 로그인

→ 오픈 날 아침: 로그인이 풀린 손님 다수

→ 10시 직전 수만 명의 '소셜 계정으로 로그인'

→ 우리 서버 → 외부 인증 서버 확인

→ 느리면 예매 페이지 진입 불가, 오픈 시각을 로그인 화면에서



© 한상훈 NestJS 백엔드 마스터

문자 발송사

→ 예매 확정 → 예매 완료 문자 → 발송 대행사 API

→ 문자 실패로 예매 취소 불가: 좌석은 정상 구매

결제 실패

예매 자체가 흔들림

문자 실패

예매 유지

→ 호출마다 다른 실패의 무게와 실패 시 동작

© 한상훈 NestJS 백엔드 마스터

목록 둘의 첫 버전

부르는 곳	실패 모양	실패 시 동작
결제사	무응답, 지연, 성공 뒤 응답 유실	?
외부 인증 서버	지연, 무응답	?
문자 발송사	지연, 거절	?

→ "3초 뒤 포기"를 적어도 남는 질문: 포기 뒤 좌석 처리

→ 빈칸: 잊지 않을 질문

© 한상훈 NestJS 백엔드 마스터

목록 셋: 자원 한도

서버가 쓰는 자원 중 숫자로 한도가 정해진 것

© 한상훈 NestJS 백엔드 마스터

커넥션 풀로 50만 개

항목	값
풀 10개, 쿼리 20밀리초	초당 500번
50만 개 처리	1,000초 = 16분 40초
마지막 응답	10시 17분쯤

→ 그사이 좌석 매진, 기다리던 손님의 새로고침

데이터 블록: 계산용 가정값

© 한상훈 NestJS 백엔드 마스터

서버 100대

항목	값
서버 100대 × 풀 10개	초당 5만 번
50만 개 처리	10초
DB가 받는 연결	1,000개

→ 새 한도: DB의 연결 1,000개, 초당 쿼리 5만 번

→ 한도 하나를 늘리면 그 뒤의 한도가 다음 줄로

© 한상훈 NestJS 백엔드 마스터

결제사 초당 200건

항목	값
결제사 처리량	초당 200건 (가정값)
5만 석 결제	250초 = 4분 남짓
좌석 맡아 두기 제한 시간	2분 (가정값)

→ 결제 줄 4분 > 제한 시간 2분 → 뒤쪽 손님의 좌석이 결제 전에 풀림

© 한상훈 NestJS 백엔드 마스터

문자 초당 100통

보낼 문자

예매 완료 문자 5만 통

초당 100통 (가정값)

500초, 8분 이상

손님 대기 불필요

문자가 늦어도 예매는 완료

© 한상훈 NestJS 백엔드 마스터

기다리는 요청의 메모리

항목	값
결제사 멈춤 5분	기다리는 요청 15만 개
요청 하나의 메모리	50KB (가정값)
15만 개	7.5GB
서버 한 대 메모리 4GB	3분 안에 한도

→ 동시에 붙들 수 있는 요청 수 = 메모리 ÷ 요청 하나의 크기

© 한상훈 NestJS 백엔드 마스터

목록 셋의 첫 버전

자원	한도 (가정값)	닿는 때
커넥션 풀	초당 쿼리 500번	오픈 첫 1초
결제사 처리량	초당 200건	결제 줄
문자 발송	초당 100통	예매 확정 직후
서버 메모리	4GB	결제사 멈춤

→ 평소에는 안 보이고 닿는 순간에만 보이는 숫자

© 한상훈 NestJS 백엔드 마스터

세 목록의 첫 버전

동시 요청이 부딪히는 곳

같은 좌석, 남은 좌석 수, 같은 손님의 결제 두 번

외부 호출과 실패 시 동작

결제사의 세 가지 실패, 소셜 로그인, 문자 발송사

자원 한도

커넥션 풀, 결제사 처리량, 문자 발송 한도, 서버 메모리

→ 기능 추가마다 늘어나는 줄 (인증 → 인증 서버 호출, 캐시 → 캐시 서버 메모리)

© 한상훈 NestJS 백엔드 마스터

세 목록과 로컬 감각

목록	깨지는 로컬 감각
동시 요청이 부딪히는 곳	요청은 하나씩 온다
외부 호출과 실패 시 동작	네트워크는 늘 답한다
자원 한도	내 노트북에서 한 번도 닿아 본 적 없는 숫자

© 한상훈 NestJS 백엔드 마스터

docs 폴더

```
ticketing/
 └──── docs/
     ├──── contention.md        동시 요청이 부딪히는 곳
     ├──── external-calls.md    외부 호출과 실패 시 동작
     └──── resource-limits.md   자원 한도
```

→ 회의 한 번으로 끝나면 오픈 날 아무도 기억 못 함

→ 새 팀원이 첫날 읽는 "이 서비스에서 위험한 것"

© 한상훈 NestJS 백엔드 마스터

오픈 순간의 목표

"오픈 순간에 서버가 할 일은 5만 명을 성공시키고, 나머지 45만 명에게 빨리, 정확하게 매진을 알리는 겁니다."

→ 돈은 빠졌는데 좌석이 없는 손님: 0명